a) Investigación:

➢ **¿Qué es la arquitectura basada en Microservicios? ¿cuál es la diferencia con la arquitectura cliente servidor monolítica? ¿Cuáles son sus ventajas y desventajas? ¿Cuáles propuestas comerciales conocidas de implementación para microservicios existen (Pagas/Open Source) en el mercado? realizar un cuadro comparativo. Describir de forma gráfica el funcionamiento de una aplicación basada en microservicios.**

Es un estilo de diseño donde una aplicación se construye como un **conjunto de servicios pequeños e independientes**. Cada uno resuelve una función de negocio concreta y tiene estas características:

- Se **desarrolla, despliega y escala por separado**.
- Tiene **su propia base de datos**.
- Se comunica con los demás mediante **APIs** o **mensajes/eventos**.
- Puede escribirse en un lenguaje distinto al de los otros servicios.

Diferencias principales entre arquitectura monolítica y arquitectura de microservicios

1. Estructura: en la arquitectura monolítica, la aplicación es una sola unidad que contiene todos los módulos juntos (interfaz, lógica de negocio y acceso a datos). En microservicios, la aplicación se divide en muchos servicios pequeños e independientes, cada uno con una función de negocio concreta.
2. Despliegue: en un monolito se publica toda la aplicación completa. En microservicios cada servicio se despliega por separado.
3. Base de datos: el monolito normalmente usa una única base de datos compartida por todos los módulos. En microservicios cada servicio tiene su propia base de datos.
4. Escalabilidad: en un monolito se debe escalar la aplicación completa. En microservicios se escala únicamente el servicio que lo necesita.
5. Tecnología: el monolito utiliza un único lenguaje y stack tecnológico para todo el sistema. En microservicios cada servicio puede usar el lenguaje y las herramientas más adecuados para su tarea.
6. Tolerancia a fallos: en un monolito, un error grave en un módulo puede hacer caer toda la aplicación. En microservicios, el fallo queda aislado en un servicio y el resto del sistema puede seguir funcionando, si el diseño es correcto.
7. Comunicación interna: en el monolito los módulos se comunican mediante llamadas a funciones dentro del mismo proceso, que son rápidas y fiables. En microservicios los servicios se comunican por red, lo que agrega latencia y posibles fallos de red.

Ventajas

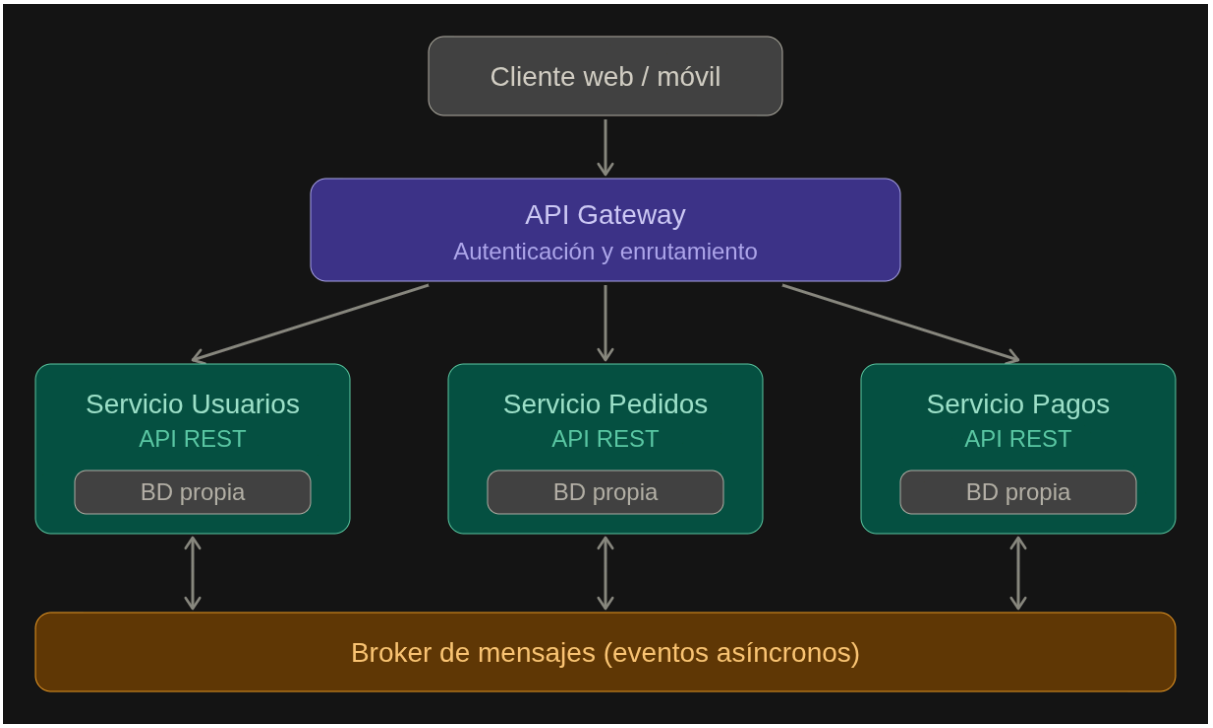
- **Escalabilidad selectiva:** se escala solo la parte que recibe más carga, lo que ahorra recursos.
- **Despliegues independientes:** se publican cambios con más rapidez y menos riesgo.

- **Resiliencia:** la caída de un servicio no tiene por qué derribar todo.
- **Libertad tecnológica:** se elige la herramienta más adecuada para cada problema.
- **Equipos autónomos:** cada equipo trabaja sin pisarse con los demás.
- **Mantenimiento más acotado:** cada código base es pequeño y más fácil de entender.
- **Encaja con CI/CD** y con la nube.

Desventajas

- **Complejidad operativa:** hay que gestionar decenas de servicios, redes, versiones y configuraciones.
- **Latencia y fallos de red:** las llamadas entre servicios son más lentas y menos fiables que las internas.
- **Consistencia de datos:** al no haber una base común, las transacciones entre servicios son difíciles.
- **Pruebas y depuración más difíciles:** seguir una petición a través de varios servicios requiere herramientas de trazabilidad.
- **Costo inicial alto:** hace falta infraestructura, automatización y madurez en DevOps.
- **Seguridad más amplia:** hay más puntos de entrada y comunicaciones que proteger.
- **Riesgo de diseño:** si se dividen mal los servicios, se obtiene un "monolito distribuido", con lo peor de ambos mundos.

Docker, Kubernetes, NGINX, Spring Cloud, Apache Kafka y varios servicios de AWS o Azure son productos que permiten orquestar una arquitectura de microservicios actualmente en el mercado.



➢ **¿Qué es un Gateway en la tecnología de microservicios y para qué se utiliza?**

Un **API Gateway** es un componente que actúa como **punto de entrada único** a un sistema de microservicios. Los clientes (web, móvil, otras aplicaciones) no hablan directamente con cada servicio. Le envían todas sus peticiones al gateway, y este las dirige al servicio correcto.

Para qué se utiliza

1. **Enrutamiento:** recibe una petición como /pedidos/123 y la envía al servicio de Pedidos. El cliente no necesita conocer cuántos servicios existen ni dónde están.
2. **Autenticación y autorización:** verifica la identidad del usuario (por ejemplo, validando un JWT) en un solo lugar, en vez de repetirlo en cada servicio.
3. **Seguridad:** filtra tráfico malicioso, aplica HTTPS/TLS y oculta la red interna del exterior.
4. **Límite de uso (*rate limiting*):** restringe la cantidad de peticiones por usuario o por IP, para evitar abusos y sobrecargas.
5. **Balanceo de carga:** reparte las peticiones entre varias instancias de un mismo servicio.
6. **Agregación de respuestas:** reúne datos de varios servicios y los devuelve al cliente en una sola respuesta, lo que reduce el número de llamadas.
7. **Traducción de protocolos y formatos:** por ejemplo, recibe REST desde el cliente y habla gRPC con los servicios internos.
8. **Caché:** guarda respuestas frecuentes para responder más rápido y aliviar a los servicios.
9. **Monitoreo y registro:** centraliza logs, métricas y trazas de todo el tráfico entrante.
10. **Versionado de APIs:** permite mantener varias versiones de la API disponibles al mismo tiempo.

➢ **¿Qué es un balanceador de carga y para qué sirve? ¿Qué relación tiene un balanceador de carga con la arquitectura de microservicios?**

Un **balanceador de carga** es un componente, de hardware o de software, que **reparte el tráfico entrante entre varios servidores o instancias** que ofrecen el mismo servicio. Los usuarios se conectan a una sola dirección, y el balanceador decide a cuál de las instancias enviar cada petición.

En una arquitectura de microservicios el balanceador es una pieza fundamental, porque **cada servicio suele ejecutarse en varias instancias** al mismo tiempo:

- **Escalado individual:** si el servicio de Pedidos recibe mucha carga, se levantan, por ejemplo, 5 instancias. El balanceador reparte las peticiones entre ellas.
- **Tolerancia a fallos:** si una instancia cae, el tráfico se redirige automáticamente a las que siguen sanas.

- **Instancias cambiantes:** en entornos con contenedores, las instancias nacen y mueren constantemente y cambian de dirección. El balanceador trabaja junto con el **descubrimiento de servicios** para conocer en cada momento cuáles están disponibles.
- **Dos niveles:**
    - **Balanceo externo:** delante del API Gateway, para repartir el tráfico entre varias instancias del propio gateway y evitar que sea un punto único de fallo.
    - **Balanceo interno:** entre los servicios, ya sea mediante un balanceador dedicado, un *service mesh* (Istio, Linkerd) o el que incluye Kubernetes (los objetos *Service* e *Ingress*).
- **Despliegues seguros:** permite estrategias como *canary* o *blue-green*, enviando solo un pequeño porcentaje del tráfico a la versión nueva para probarla antes de generalizarla.

c) Investigación:

➢ ¿Qué es Spring Cloud? ¿Cuáles son sus ventajas? ¿Qué mejoras aplica para la programación de microservicios?

Spring Cloud es un conjunto de herramientas del ecosistema Spring (Java) que ayuda a construir sistemas distribuidos y microservicios. No es un único producto, sino una colección de proyectos que se apoyan en Spring Boot y resuelven los problemas típicos de esta arquitectura: configuración centralizada, descubrimiento de servicios, comunicación entre ellos, tolerancia a fallos, seguridad y trazabilidad.

La idea es que el programador se concentre en la lógica de negocio, y que Spring Cloud aporte los patrones comunes ya implementados y listos para usar, casi siempre con anotaciones y configuración.

Ventajas

- **Productividad:** muchos patrones complejos se activan con una anotación y un poco de configuración.
- **Integración natural con Spring Boot:** aprovecha lo que el equipo ya conoce, sin curva de aprendizaje desde cero.
- **Estandarización:** todos los servicios resuelven los problemas comunes de la misma forma.
- **Abstracción:** se puede cambiar de Eureka a Consul, o de Kafka a RabbitMQ, con pocos cambios en el código.
- **Ecosistema maduro y de código abierto:** licencia Apache 2.0, gran comunidad y abundante documentación.
- **Resiliencia incorporada:** circuit breakers, reintentos y límites de tiempo vienen listos para usar.
- **Compatible con la nube y los contenedores:** funciona con Docker y Kubernetes, y en cualquier proveedor.

Mejoras que aporta a la programación de microservicios

1. **Menos código repetitivo:** no hay que programar a mano el registro de servicios, el balanceo ni los reintentos.
2. **Configuración externalizada:** se puede cambiar la configuración sin recompilar ni redesplegar, y mantener valores distintos por entorno (desarrollo, pruebas, producción).
3. **Llamadas entre servicios más simples:** con OpenFeign se declara una interfaz y Spring genera la llamada HTTP.
4. **Mayor estabilidad:** el circuit breaker evita que la caída de un servicio provoque una cascada de fallos.
5. **Mejor visibilidad:** la trazabilidad distribuida y las métricas ayudan a encontrar dónde falla una petición.
6. **Seguridad centralizada:** la autenticación (por ejemplo con OAuth2 y JWT) puede resolverse en el gateway.
7. **Escalabilidad más fácil:** como los servicios se registran dinámicamente, agregar instancias no exige cambios manuales.
8. **Pruebas más confiables:** las pruebas de contrato detectan incompatibilidades entre servicios antes de llegar a producción.